

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0601 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15 Total Marks:25 Annexure A

Experiments

Code of Conduct:

I N.HarshaVardhan(192324189)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N.HarshaVardhan

Annexure A

Short Answer Test

1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

Answer:

Naïve Bayes Classifier – Feature Independence and Laplace Smoothing

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score,
recall_score iris = load_iris()
X = iris.data
y = iris.target

feature_sets = {
    "Sepal Features": [0, 1],
    "Petal Features": [2, 3],
    "All Features": [0, 1, 2, 3]
}

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

for name, features in feature_sets.items():

    model = GaussianNB()

    model.fit(X_train[:, features], y_train)

    y_pred = model.predict(X_test[:, features])

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test, y_pred, average="weighted"
    )
    recall = recall_score(
        y_test, y_pred, average="weighted"
    )

    print("\n", name)
    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall :", round(recall, 4))
```

OUTPUT:

```

***
    Sepal Features
    Accuracy : 0.6889
    Precision: 0.6795
    Recall   : 0.6889

    Petal Features
    Accuracy : 0.9333
    Precision: 0.9345
    Recall   : 0.9333

    All Features
    Accuracy : 0.9111
    Precision: 0.9155
    Recall   : 0.9111

```

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

Answer:

Maximum Likelihood Estimation vs Bayesian Estimation

Program:

```

import numpy as np
np.random.seed(42)
true_mean = 50
true_variance = 25
true_std = np.sqrt(true_variance)
sample_sizes = [10, 50, 500]
prior_mean = 45
prior_variance = 100

for n in sample_sizes:
    data = np.random.normal(
        true_mean,
        true_std,
        n
    )
    mle_mean = np.mean(data)
    mle_variance = np.var(data)
    posterior_mean = (
        (n * mle_mean / true_variance)
        + (prior_mean / prior_variance)
    ) / (
        (n / true_variance)
        + (1 / prior_variance)
    )

```

```

print("\nSample Size:", n)
print("MLE Mean:", round(mle_mean, 3))
print("MLE Variance:", round(mle_variance, 3))
print("Bayesian Mean:", round(posterior_mean, 3))

```

OUTPUT:

```

***
Sample Size: 10
MLE Mean: 52.24
MLE Variance: 11.762
Bayesian Mean: 52.864

Sample Size: 50
MLE Mean: 48.624
MLE Variance: 19.818
Bayesian Mean: 48.686

Sample Size: 500
MLE Mean: 50.801
MLE Variance: 23.81
Bayesian Mean: 49.999

```

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

Answer:

Expectation-Maximization Algorithm for Mixture Models

Program:

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture

np.random.seed(42)

c1=np.random.normal([2,2],0.6,(100,2))
c2=np.random.normal([7,7],0.6,(100,2))
c3=np.random.normal([2,7],0.6,(100,2))
X=np.vstack([c1,c2,c3])

model=GaussianMixture(n_components=3,max_iter=1,warm_start=True,random_state=42

) likelihoods=[]

for i in range(30):
    model.fit(X)
    likelihoods.append(model.lower_bound_)

plt.figure(figsize=(8,5))
plt.plot(range(1,31),likelihoods,marker="o",color="blue")
plt.xlabel("EM Iteration")
plt.ylabel("Log-Likelihood")
plt.title("EM Convergence")
plt.grid(True)

```

```

plt.show()

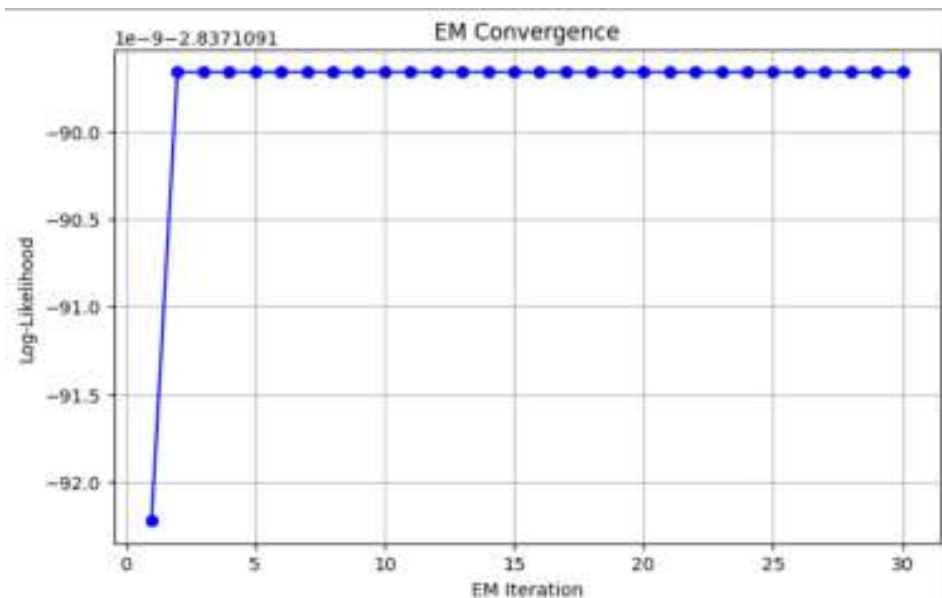
clusters=[2,3,4,5]
likelihood=[]

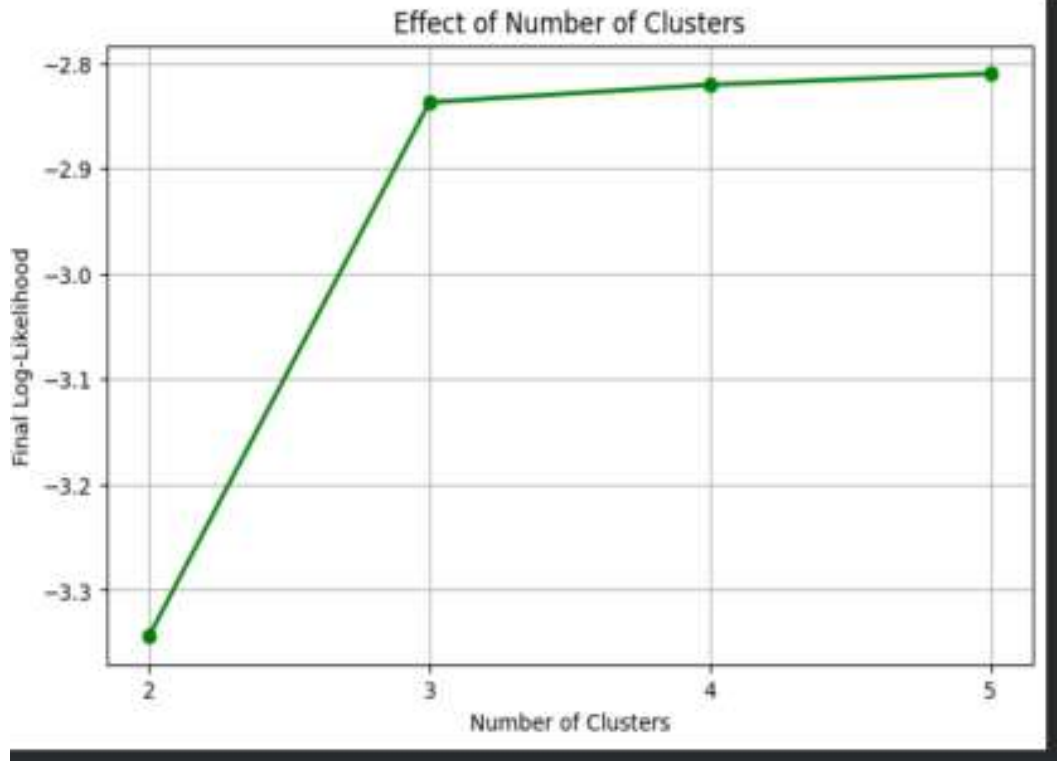
for k in clusters:
    model=GaussianMixture(n_components=k,max_iter=100,n_init=10,random_state=42
    ) model.fit(X)
    likelihood.append(model.lower_bound_)

plt.figure(figsize=(8,5))
plt.plot(clusters,likelihood,marker="o",linewidth=2,color="green")
plt.xlabel("Number of Clusters")
plt.ylabel("Final Log-Likelihood")
plt.title("Effect of Number of Clusters")
plt.xticks(clusters)
plt.grid(True)
plt.show()

```

OUTPUT:





4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

Answer:

Bayesian Belief Network and Probabilistic Inference

Program:

```
import numpy as np
import matplotlib.pyplot as plt

P_D = 0.20
P_F_given_D = 0.85
P_F_given_notD = 0.10
P_C_given_D = 0.75
P_C_given_notD = 0.20
P_T_given_D = 0.70
P_T_given_notD = 0.15

def posterior(evidence):
    pd = P_D
    pn = 1 - P_D
    for symptom, value in evidence.items():
        if symptom == "Fever":
            a, b = P_F_given_D, P_F_given_notD
        elif symptom == "Cough":
            a, b = P_C_given_D, P_C_given_notD
        elif symptom == "Fatigue":
            a, b = P_T_given_D, P_T_given_notD
```

```

if value:
    pd *= a
    pn *= b
else:
    pd *= 1 - a
    pn *= 1 - b
return pd / (pd + pn)

evidence_cases = {
    "No Evidence": {},
    "Fever": {"Fever": True},
    "Cough": {"Cough": True},
    "Fever+Cough": {"Fever": True, "Cough": True},
    "Fever+Cough+Fatigue": {"Fever": True, "Cough": True, "Fatigue": True},
    "No Fever": {"Fever": False}
}
results = {k: posterior(v) for k, v in evidence_cases.items()}

plt.figure(figsize=(9,5))
plt.bar(results.keys(), results.values(), color="steelblue")
plt.ylabel("P(Disease | Evidence)")
plt.xlabel("Evidence Condition")
plt.title("Posterior Probability Under Different Evidence")
plt.xticks(rotation=25)
plt.ylim(0,1)
plt.grid(axis="y",alpha=0.3)
plt.tight_layout()
plt.show()

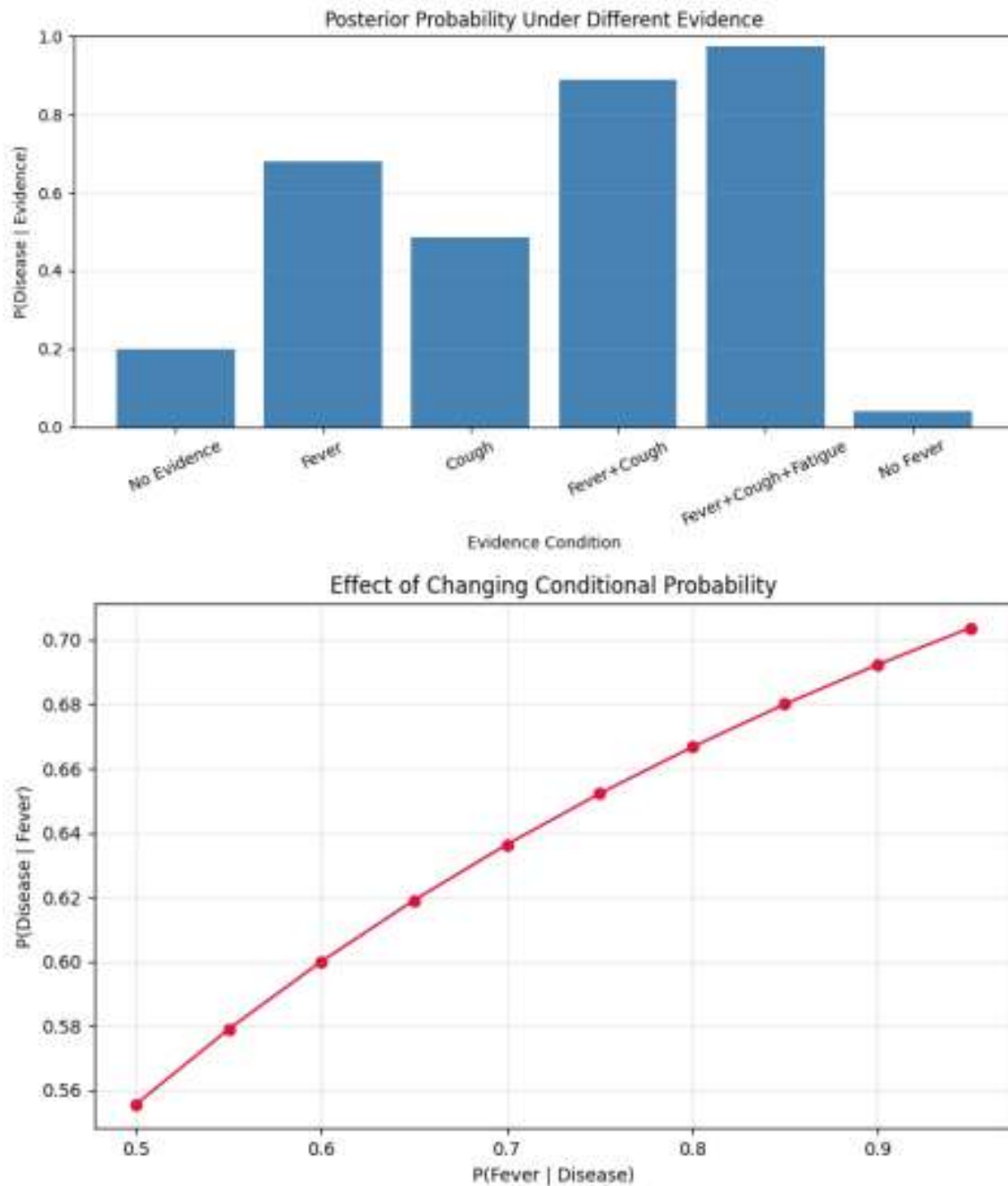
fever_values = np.linspace(0.5,0.95,10)
modified_results = []

for p in fever_values:
    P_F_given_D = p
    modified_results.append(posterior({"Fever": True}))

plt.figure(figsize=(8,5))
plt.plot(fever_values, modified_results, marker="o", color="crimson")
plt.xlabel("P(Fever | Disease)")
plt.ylabel("P(Disease | Fever)")
plt.title("Effect of Changing Conditional Probability")
plt.grid(True,alpha=0.3)
plt.tight_layout()
plt.show()

```

OUTPUT:



5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

Answer:

Concept Learning Using Candidate Elimination

Program:

```
import pandas as pd
```

```
data = [
```



```

['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

```

```

columns = [
'Sky', 'AirTemp', 'Humidity',
'Wind', 'Water', 'Forecast', 'EnjoySport'
]

```

```
df = pd.DataFrame(data, columns=columns)
```

```

print("Training Data:")
print(df)

```

```
def candidate_elimination(df):
```

```

    X = df.iloc[:, :-1].values
    Y = df.iloc[:, -1].values

```

```
    n = len(X[0])
```

```

    S = ['0'] * n
    G = [['?'] * n]

```

```
    for i in range(len(X)):
```

```

        example = X[i]
        label = Y[i]

```

```
        if label == 'Yes':
```

```

            G = [
                g for g in G
                if all(
                    g[j] == '?' or g[j] == example[j] for j in
                    range(n)
                )
            ]

```

```
        for j in range(n):
```

```

            if S[j] == '0':
                S[j] = example[j]

```

```

            elif S[j] != example[j]:
                S[j] = '?'

```

```
        else:
```

```
            new_G = []
```

```
            for g in G:
```

```
                if all(
```

```

    g[j] == '?' or g[j] == example[j] for j in
range(n)
):

for j in range(n):

    if g[j] == '?' and S[j] != example[j]: new_g =
g.copy()
    new_g[j] = S[j]
    new_G.append(new_g)

else:
    new_G.append(g)

G = new_G

return S, G

```

S, G = candidate_elimination(df)

print("\nFinal Specific Boundary
(S):") print(S)

print("\nFinal General Boundary
(G):") for g in G:
 print(g)
OUTPUT:

```

*** Training Data:
      Sky AirTemp Humidity   Wind Water Forecast EnjoySport
0  Sunny   Warm   Normal  Strong  Warm   Same       Yes
1  Sunny   Warm    High  Strong  Warm   Same       Yes
2  Rainy   Cold    High  Strong  Warm  Change      No
3  Sunny   Warm    High  Strong  Cool   Change      Yes

Final Specific Boundary (S):
['Sunny', 'Warm', '?', 'Strong', '?', '?']

Final General Boundary (G):
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
['?', '?', '?', '?', '?', '?']

```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 4 – Exemplary (4 Marks)

Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand